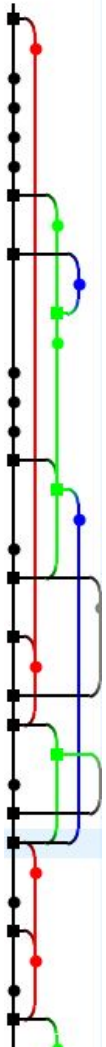
Git at CGS

Best practices

Adapted from Dillon Walton's brown bag series talk by:
Andy Miller
Nishant Gaonkar



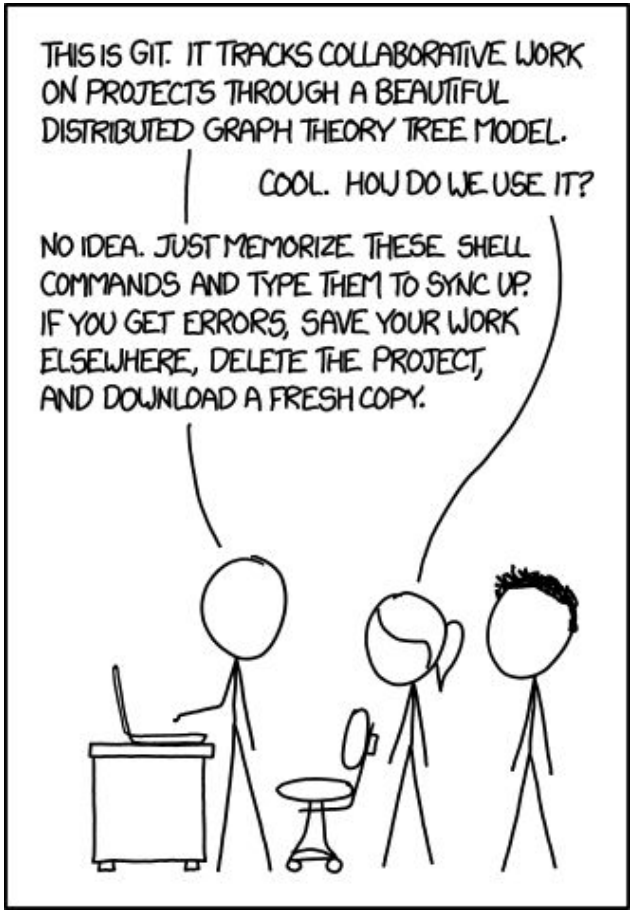
Why use git

- Collaborate with others well on code
- Keep track of your own work
- Have multiple versions of code which can be compared and merged together

https://xkcd.com/1597/

Why use git

- Collaborate with others well on code
- Keep track of your own work
- Have multiple versions of code which can be compared and merged together



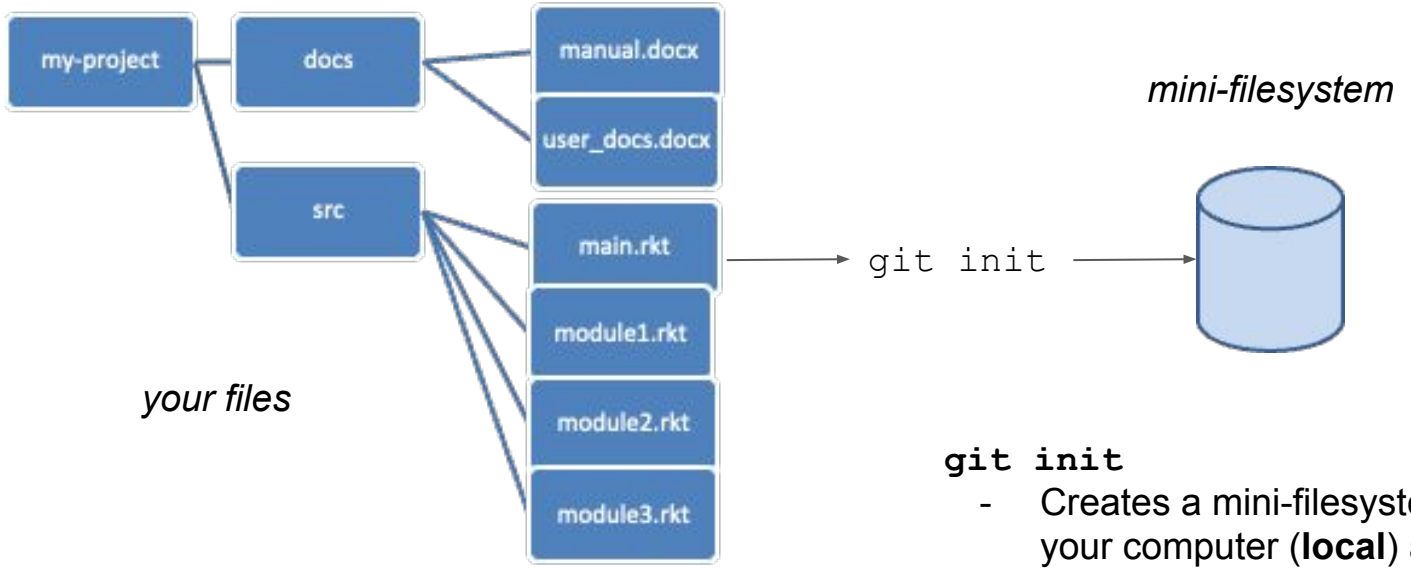
https://xkcd.com/1597/

What is Git & why do we use it?

1. Before git
    a. Think - version control with excel files
    b. Problem: MANUAL (we have to manually resolve differences in files, which can cause extreme complications)
2. What is git?
    a. Git is a *version control system* - a software program which manages version control
        i. Works as a “time machine” for going back to earlier versions of your code + provides great support for different versions of the same project
3. Why do we use it?
    a. Track changes to code over time
        i. Easily roll back changes to code if something goes wrong
    b. Simplifies concurrent work and merging of changes from many people into the same project

Table of Contents

Core concepts in git: `git init`



`git init`

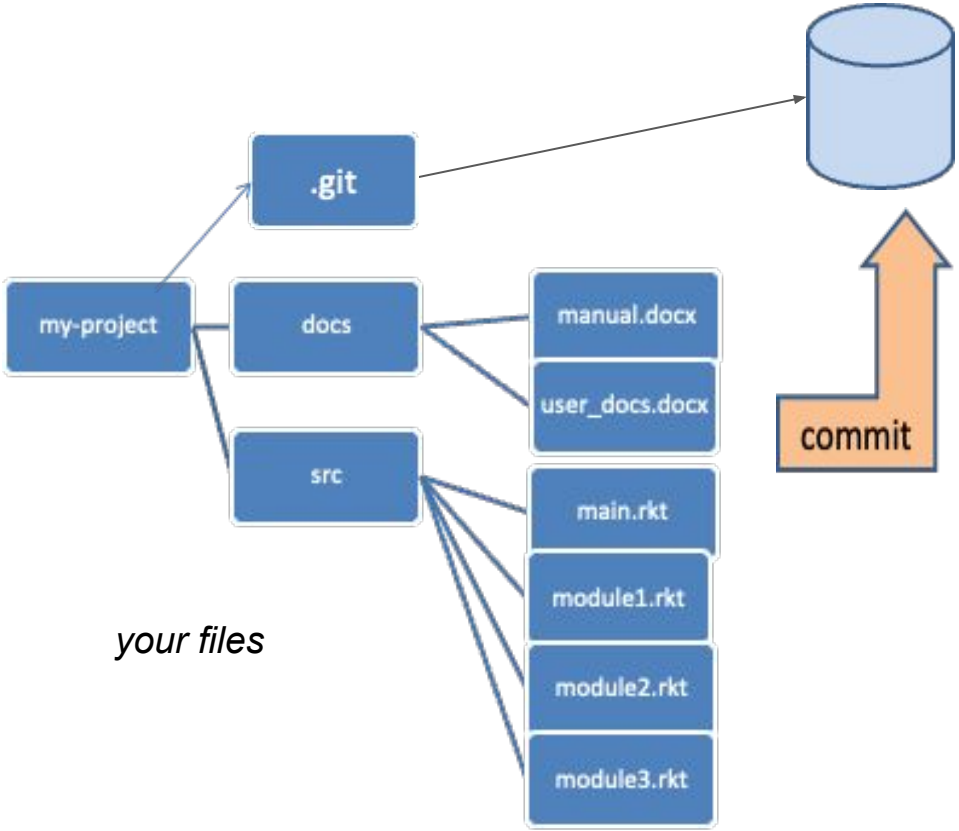
- Creates a mini-filesystem which sits on your computer (**local**) and keeps track of your files
- **this is all local**
- Many people will use git locally as a pure version control system

Git init

Git clone

- Git init & setting up a link to a remote & pulling in the entire remote

Core concepts in git: `git add + git commit`



mini-filesystem

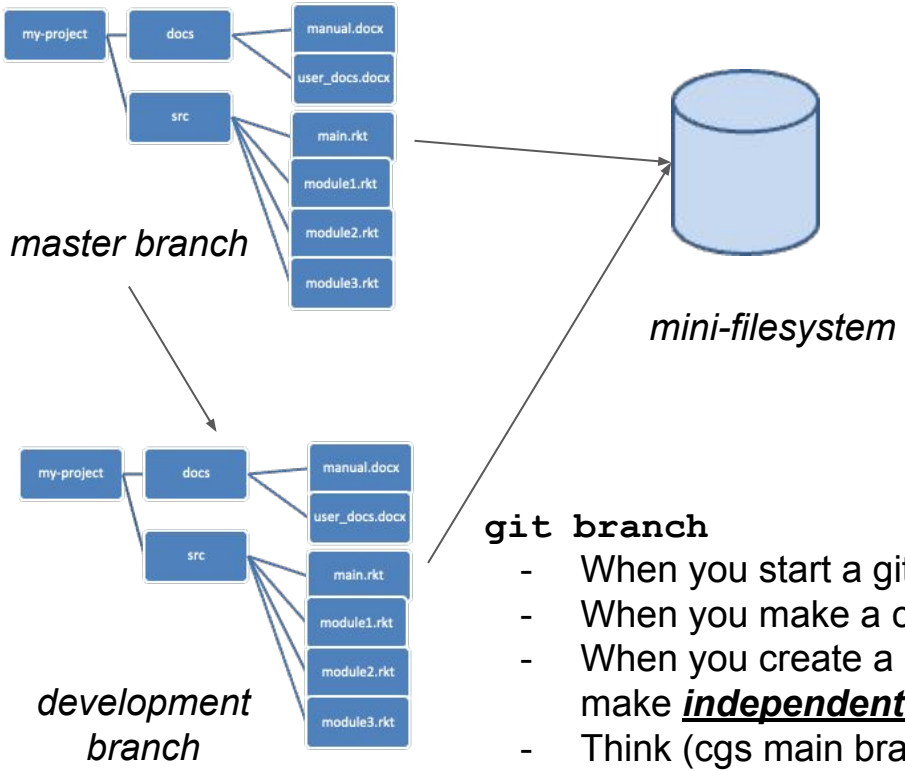
`git add`

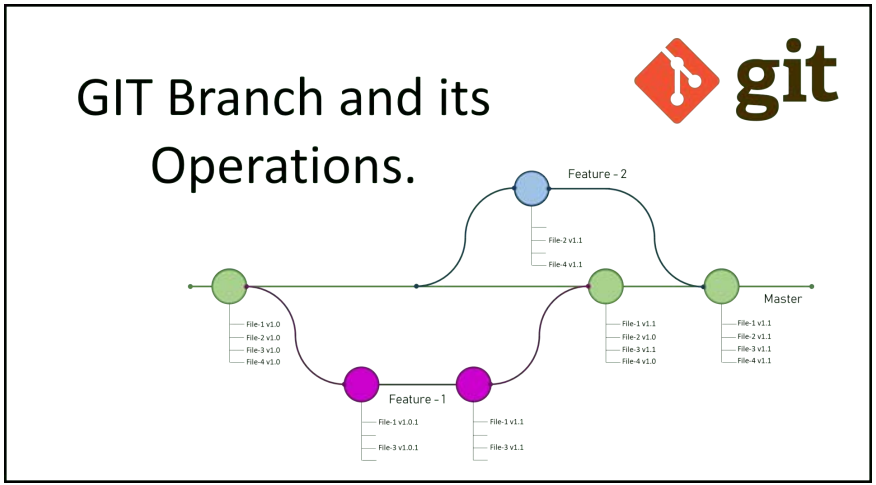
- Relevant in cases where you have created a new file
- You must “add” that file to the “staging” area → in other words, you are telling the git software that you want changes to this file to be tracked

`git commit`

- You are recording all local changes to tracked files into the mini-filesystem
- The mini-filesystem ***appends*** changes, it does not overwrite - therefore everything you ever commit can be recovered (this is the genius of the system)
- Each commit is a version of your code that you can revert back to

Core concepts in git: `git branch`

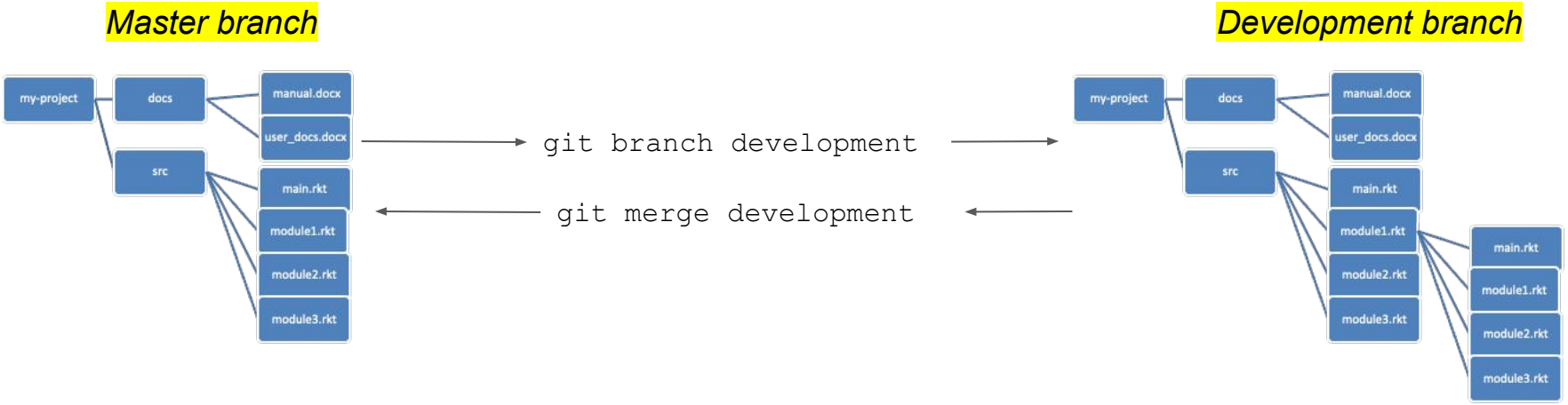




`git branch`

- When you start a git repository, you start with a default branch
- When you make a commit → you are making changes on a branch
- When you create a new branch, you are creating a copy of your files to make ***independent commits*** to, ***therefore you edits are isolated***
- Think (cgs main branch, cgs study2 branch) → they are independent
- In git, you are always on a branch: `git status` tells you which one

Core concepts in git: `git merge`



`git merge`

- Combines changes from one branch into another.
- The “direction” of the merge is important as you want to ***bring in*** new changes → the branch you are on is the branch you are merging into

- Changes to the same code in the same file by both branches give rise to a **merge conflict** → solve by reviewing the conflicts one by one
- To avoid challenges in this stage, frequently commit your changes on your development branch and pull in changes by other branches to quickly identify conflicts

Core concepts in git: Local vs Remote

Local

- Everything up to this point has been **local**, meaning isolated to your computer and unknown to anyone else
- **Local** means you are interacting with the mini-filesystem ***on your computer***
- Most of the work you do using the git software is local (init, branch, add + commit)
- In fact, many users of git use the software entirely in this way, and it is incredibly useful

Remote

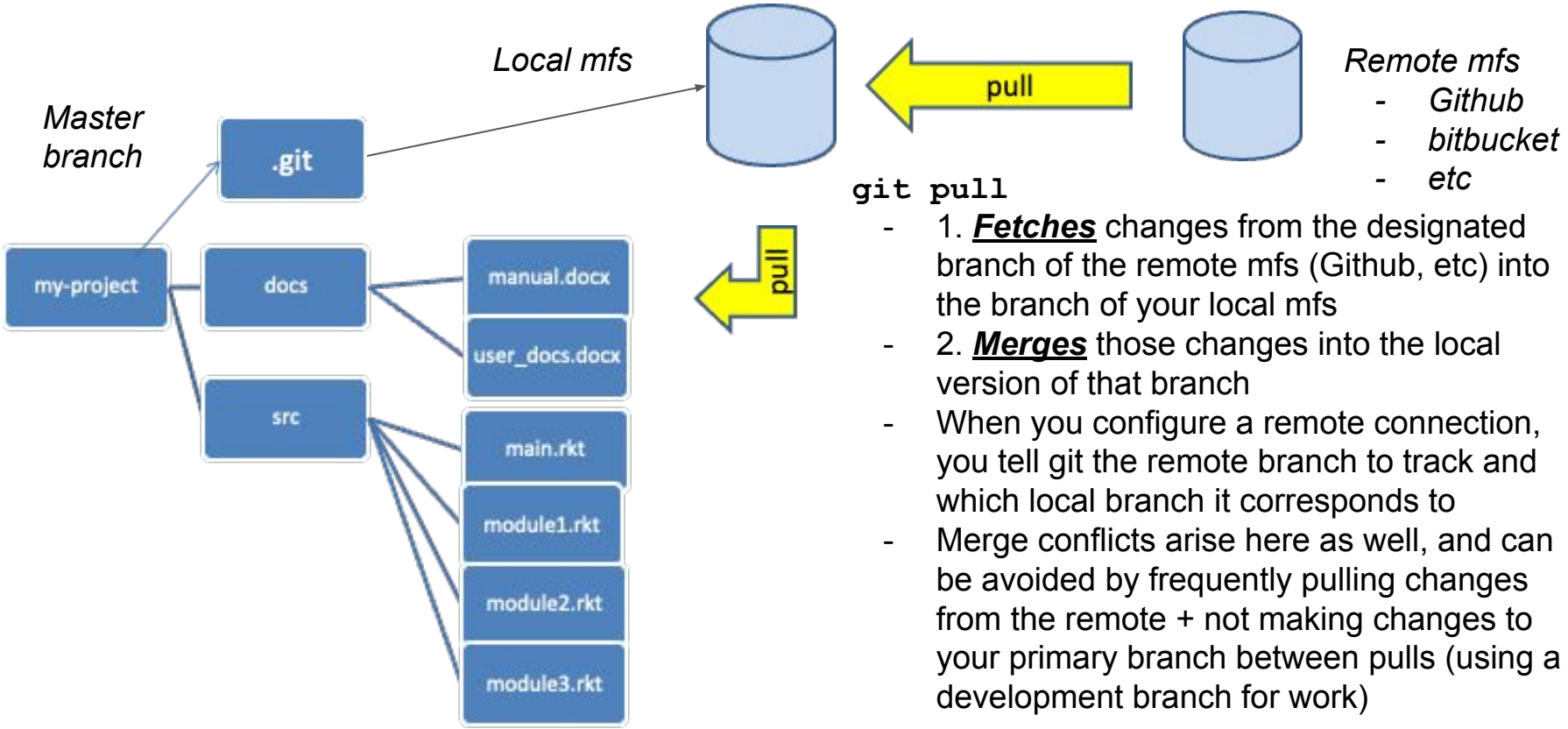
- **Remote**, meaning far away and seen by everyone
- **Remote** means you are interacting with the mini-filesystem ***on a remote server***
- Think of **remote** as a way to save your **local** mini-filesystem to the cloud
- The only operations interacting with remote is pushing changes from your **local mfs** to the **remote mfs** & pulling changes from the **remote mfs** into your **local mfs**
- Because multiple people can push and pull from the same **remote mfs**, you are easily able to integrate changes from other people into your **local mfs**

Core concepts in Git: `git pull` and `git push`

- Interacting with the remote repo (on Github)

Illustrations from the Openscapes blog *GitHub for supporting, contributing, and failing safely* by Allison Horst and Julia Lowndes.

Core concepts in Git: `git pull`



`git pull`

- 1. ***Fetches*** changes from the designated branch of the remote mfs (Github, etc) into the branch of your local mfs
- 2. ***Merges*** those changes into the local version of that branch
- When you configure a remote connection, you tell git the remote branch to track and which local branch it corresponds to
- Merge conflicts arise here as well, and can be avoided by frequently pulling changes from the remote + not making changes to your primary branch between pulls (using a development branch for work)

Understanding Git Pull and Fetch + Merge

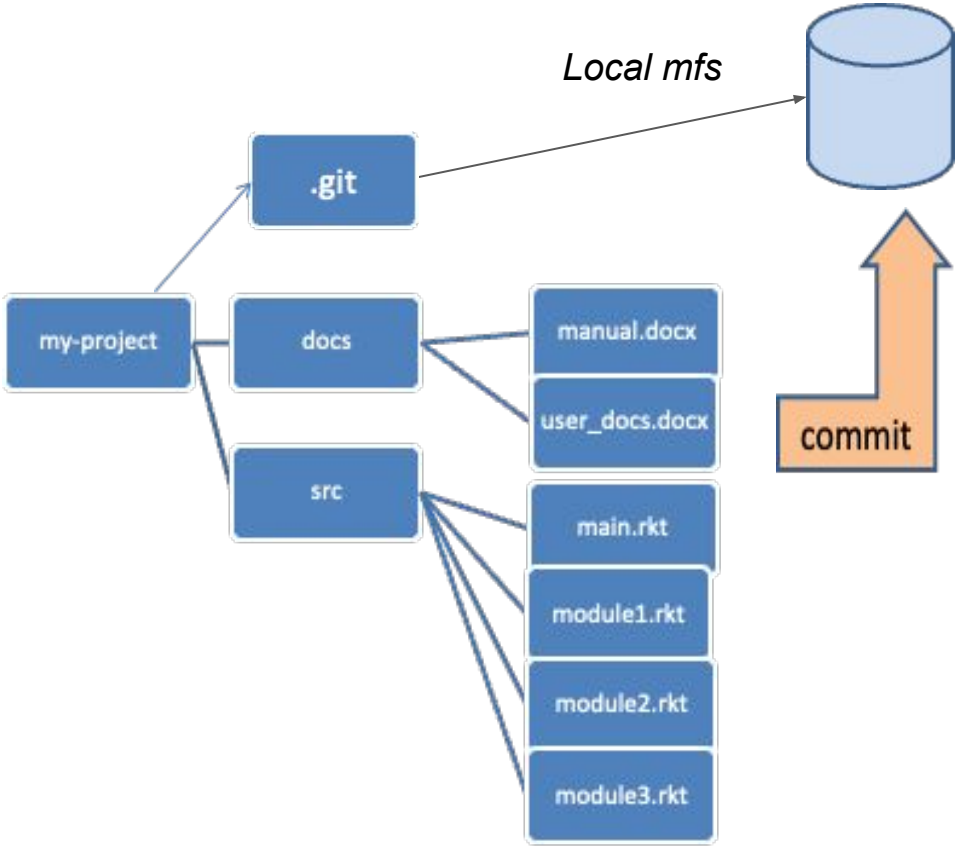
- Git Pull:
  - Combines two operations: fetch and merge
  - Command: `git pull origin branch_name`
  - Automatically fetches and merges remote branch into current branch
- Git Fetch + Git Merge:
  - Step 1: Fetch the Changes:
    - Only downloads new data from remote repository
    - Does not integrate changes into your local branch
    - Command: `git fetch origin branch_name`
  - Step 2: Merge the Changes:
    - Manually merge fetched changes into your current branch
    - Provides control over when to integrate
    - Command: `git merge origin/branch_name`

Understanding Git Pull and Fetch + Merge

- Git Pull:
  - Combines two operations: fetch and merge
  - Command: `git pull origin branch_name`
  - Automatically fetches and merges remote branch into current branch
- Git Fetch + Git Merge:
  - Step 1: Fetch the Changes:
    - Only downloads new data from remote repository
    - Does not integrate changes into your local branch
    - Command:
      `git fetch origin branch_name`
  - Step 2: Merge the Changes:
    - Manually merge fetched changes into your current branch
    - Provides control over when to integrate
    - Command:
      `git merge origin/branch_name`

Illustrations from the Openscapes blog *GitHub for supporting, contributing, and failing safely* by Allison Horst and Julia Lowndes.

Core concepts in git: `git push`



`git push`

- ***Pushes*** the branch commits on your local mfs to a remote mfs, which others can pull from
- You will not be able to push if your mfs contains conflicts with the remote mfs. Therefore it is important that you pull commits from the remote *and then* push your commits, to ensure you've integrated commits from others first
- Once you push successfully, the remote version of the branch will be an exact copy of your local version

Best Practices

1. Use the terminal (terminal on mac or git bash on windows)
    a. git features provided by RStudio are simplifications of the software and using them can lead to unintended problems

1. Always start a new (local) branch to do modifications to
    b. This can be called '`yourName_primaryBranchName` ', such as `andy_main` or `andy_study2`

1. Commit your changes after each notable point in your development
    b. Make sure the message you add to your commit is clear, so if you need to revert back you will understand version this commit represents (if i make this commit it will…)

1. Pull in changes from the remote frequently to keep things in sync

1. Designate the 'main' (or may be called 'master') local branch to push and pull from remote, and nothing else. After doing any development work on another branch, checkout your 'main' branch and pull in remote changes ***before*** merging your development branch
    b. The primary branch is whichever branch is being synced with a remote branch
    c. This ensures you are always in sync with the remote, which is extremely important

1. Only push changes after you have pulled in changes from remote and merged your development branch successfully
    b. Resolve merge conflicts ***locally***

1. Be descriptive with your branch names and commit messages
    b. This can save you a lot of time
    c. Commit messages complete the following sentence: "If I make this commit, it will _____". This shows the purpose of the commit
    d. For longer descriptions separate it from the short message by two 'enters' / newlines

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

Preparation: Cloning a repository

In this example: study2 branch of the practice repository

1. Download Git for Mac or Windows (can follow these Git settings during installation for Git for Windows)
2. Set the default text editor to Notepad++ or TextEdit (Mac) using the Set Default Text Editor for Git section
3. In the terminal (or Git Bash on PC), change directory to the desired location of the repository
   Alternatively, for Git Bash right-click in the folder > Show more options > Open Git Bash Here

   ```
   cd working/directory/file/path
   ```

4. Clone the repository
   Initializes a local repository (git init) → copies the remote mfs (all of the branches) of the practice repository and merges it into the empty local mfs it just created
   NOTE: Before cloning a private repository for the first time, such as in our umd-cgs GitHub, set up a personal authentication token, following **Download private repo - PAT - Github personal access token** in the Git - Personal Accxess Token etc document

   ```
   git clone https:// githubUsername:yourPAT@github.com/umd-cgs/practice
   ```

5. Enter the repository
   Note: replace `practice` with the name of the cloned repository

   ```
   cd practice
   ```

6. Switch from the main (master) branch to the branch relevant to the CGS project (eg. `study2`)
   (because git clone makes a copy of the entire remote mfs, you will now have a local version of each branch of the remote mfs as well)

   ```
   git checkout study2
   ```

7. Create a new branch **locally** and switch to it:
   This can be called 'yourName_primaryBranchName', such as dillon_main or dillon_study2

   ```
   git checkout -b dillon_study2
   ```

Workflow 1: Merging updates from others into your local branch

NOTE: this particular branch is called ‘study2’ and the personal local branch is called ‘dillon_study2’

1. Save all files locally
2. Check for updates after fetching the most recent information from the remote repository on Github
   a. `git fetch`
   b. `git log --oneline dillon_study2..origin/study2`
3. Switch to 'study2' Branch:
   a. `git checkout study2`
4. Pull Latest Changes:
   a. `git pull origin study2`
   b. (alternatively, can dado fetch and merge)
   c. may need to stash changes: `git stash`
5. Switch to your personal local branch
   a. `git checkout dillon_study2`
6. Merge in changes from the (local) updated branch into your your personal local branch
   a. `git merge study2`
   b. **Note:** If stashed changes before, then put them back in: `git stash pop`
7. Resolve Conflicts:
   a. Edit the files with conflicts, then save the files
      i. (generally keep the ‘updated upstream’ parts, unless have good reason to choose certain ‘HEAD’ or ‘stashed’ sections, the version that you have in the personal branch)
      NOTE: for conflicting areas, from <<<<<<< HEAD to ======= shows your version, and from ======= to >>>>>>> branch_name (in this case study2) shows the version that you brought in from the remote repo
   b. Test Changes to ensure functionality isn’t compromised
   c. Add those edited files which had conflicts, then commit
      i. `git add filename1 filename2`
   d. `git commit -m "Resolve merge conflicts to get updates"`

Workflow 2: Contributing to a (remote) branch

Note: This workflow is starting on the personal local branch called ‘dillon_study2’ and is contributing to the branch called ‘study2’

1. Save all files locally
2. Check Status
    a. Execute: `git status`
3. Stage Changes
    a. To stage specific files: `git add filename 1 filename_2 filename_3`
    b. To stage all files in the current and subfolders: `git add .`
    Note:: take care not to add any data files, that shouldn’t be uploaded to Github, especially any large ones, such as .Rhistory and .RData files, which can cause issues with the repo as a whole. The Git interface in **Rstudio** or **Github Desktop** can be helpful to see the changes in each file and stage (add) only the ones (or even parts of one) that are desired to be committed
4. Commit Changes with a message name
    a. Execute: `git commit -m “Update study2 industry policy scripts”`
5. Switch to Project Branch
    a. Switch to 'study2' branch: `git checkout study2`
6. Update Project Branch
    a. Pull latest changes: `git pull origin study2`
    Note: if it doesn’t let you, because your changes would be overwritten, you will likely need to stash changes: `git stash`
7. Merge Changes
    a. Merge your changes from 'dillon_study2' to 'study2': `git merge dillon_study2`
    b. Resolve any conflicts if necessary:
        i. Note: you may need to resolve differences (conflicts) at this point. git status can provide more info. Edit the files with conflicts, then save the files (generally keep the ‘HEAD’ parts, unless you have good reason to choose some of the sections from you personal branch
        ii. Stage the edited files: `git add filename1 filename2`
        iii. Ensure functionality isn’t compromised
        iv. Commit the resolution: `git commit -m "Resolve merge conflicts before pushing"`
8. Verify commits to contribute
    a. See which commits are on your local branch that aren’t on the remote: `git log --oneline origin/study2..study2`
    Note: if there were any conflicts that were resolved, then that commit will also show, which is fine
9. Push Changes
    a. For routine/small updates: `git push origin study2`
    b. For large updates/main branch:
        i. Push to a new branch and make a pull request on the Github page: `git push origin study2:dev_study2`
10. Switch Back to Personal Branch
    a. Return to 'dillon_study2': `git checkout dillon study2`
    b. Note: If stashed changes before, then put them back in: `git stash pop`
11. Merge in any updates from remote that you incorporated earlier with the pull
    a. Execute: `git merge study2`

Other helpful commands

- Git status
  - To see the list of staged files that could be committed
- Git add
  - Add files to staging to be committed
- Git diff – stat –cached [other branch]
  - To compare committed differences between the branch you are in and another branch. Helpful to use before git push, to confirm what changes will be made
- Git diff [filename]
  - To see differences between your local file and remote file, if changes are noted using git status

(Potentially) helpful GUIs

Graphic User Interface (GUI) name	**Strengths**
Sourcetree	- Lots of functionality
Git tab within Rstudio	- Easily diff the files with uncommitted changes - Very accessible when working in R, especially on R projects - Stage certain parts of files to commit
Github Desktop	- Stage certain parts of files to commit
TortoiseGit	- Compare entire branches - Dig into revision history

Note: These are supplemental tools to using the commandline, which can be used from Terminal (Mac), Git Bash (PC), and Cygwin

Resources

1. Git website
   a. On-line book
   b. Reference pages
2. Tutorials
   a. FreeCodeCamp (crash course: 1hr)
   b. Colt Steele (overview: 15 min)
   c. Colt Steele Github (overview: 20 min)
   d. Traversy Media (crash course: 30 min)
   e. Corey Schafer (crash course: 30 min)
3. Great links
   a. Bard / ChatGPT (highly suggest)
   b. Atlassian guide
   c. Baeldung guide
4. My contact information
   a. Email: amille17@umd.edu
   b. Slack: Andy Miller
   c. Github: github.com/andym331

Thank you!